Nama : Nayla Rabia Gustari
NPM : 20241320034
Matkul : Pemograman Modern Python

A. Soal Teori

1. Jelaskan perbedaan antara parameter dan argumen. Berikan contoh masingmasing dalam konteks fungsi Python.
   - Parameter adalah variabel yang ditulis di dalam tanda kurung saat mendefinisikan fungsi. Sementara itu, argumen adalah nilai yang diberikan kepada fungsi ketika fungsi tersebut dipanggil.

     Contoh dalam Python:

     ```
     def sapa(nama): # nama adalah parameter

     print("Halo,", nama)sapa("Nayla") # "Nayla" adalah argumen
     ```

     Penjelasan:

     - nama merupakan parameter karena ditulis saat fungsi sapa() dibuat.
     - "Nayla" merupakan argumen karena merupakan nilai yang diberikan saat fungsi dipanggil.
     - Hasil program adalah Halo, Nayla. Jadi, perbedaannya terletak pada penggunaannya: parameter digunakan saat mendefinisikan fungsi, sedangkan argumen diberikan saat memanggil fungsi.

2. Apa yang dimaksud dengan aturan LEGB dalam scope Python? Jelaskan setiap level dan berikan contoh situasi di mana variabel dari masing-masing level diakses.
   - LEGB adalah aturan yang digunakan Python untuk mencari variabel berdasarkan urutan scope atau ruang lingkupnya. LEGB merupakan singkatan dari Local, Enclosing, Global, dan Built-in.

     Berikut penjelasan setiap levelnya.

     a. Local (L)

     Local adalah variabel yang dibuat di dalam suatu fungsi dan hanya dapat diakses di dalam fungsi tersebut.

     ```
     def hitung():
         angka = 10
         print(angka)

     hitung()
     ```

Hasil: 10

Variabel angka termasuk Local karena dibuat di dalam fungsi hitung().

b. Enclosing (E)

Enclosing adalah variabel yang berada di dalam fungsi luar dan dapat diakses oleh fungsi yang berada di dalamnya.

```
def luar():
    pesan = "Halo"

    def dalam():
        print(pesan)

    dalam()

luar()
```

Hasil: `Halo`

Variabel `pesan` diakses oleh fungsi `dalam()` dari fungsi `luar()` yang membungkusnya.

c. Global (G)

Global adalah variabel yang dibuat di luar fungsi sehingga dapat diakses dari berbagai fungsi dalam modul yang sama.

```
nama = "Nayla"

def sapa():
    print(nama)

sapa()
```

Hasil: `Nayla`

Variabel `nama` termasuk Global karena dibuat di luar fungsi `sapa()`.

d. Built-in (B)

Built-in adalah nama fungsi atau objek bawaan Python yang dapat digunakan tanpa harus mendefinisikannya terlebih dahulu.

```
angka = [10, 20, 30]

print(len(angka))
```

Hasil: 3

Fungsi `len()` merupakan fungsi bawaan Python yang digunakan untuk menghitung jumlah elemen dalam sebuah objek.

Kesimpulan: Python mencari variabel dengan urutan Local → Enclosing → Global → Built-in. Jika variabel tidak ditemukan dalam urutan tersebut, Python akan menghasilkan error `NameError`.

3. Mengapa penggunaan objek mutable sebagai default parameter dianggap sebagai pitfall? Jelaskan mekanisme di balik perilaku ini dan berikan solusi yang benar.
   - Objek mutable adalah objek yang isinya dapat diubah setelah dibuat, contohnya list dan dictionary. Penggunaan objek mutable sebagai default parameter dianggap sebagai pitfall karena Python hanya membuat objek default tersebut satu kali saat fungsi didefinisikan, bukan setiap kali fungsi dipanggil.

     Contoh:

     ```
     def tambah_data(data=[]):
         data.append(1)
         return data

     print(tambah_data())
     print(tambah_data())
     ```

     Hasil :
     [1]
     [1, 1]

     Hal ini terjadi karena list kosong `[]` digunakan kembali pada setiap pemanggilan fungsi. Akibatnya, data dari pemanggilan sebelumnya tetap tersimpan dan dapat menyebabkan hasil yang tidak diharapkan.

     Solusi yang benar adalah menggunakan `None` sebagai default parameter, kemudian membuat list baru jika nilainya `None`.

     ```
     def tambah_data(data=None):
         if data is None:
             data = []
         data.append(1)
         return data
     ```

     Dengan cara ini, setiap pemanggilan fungsi tanpa argumen akan membuat list baru sehingga data tidak tercampur dengan pemanggilan sebelumnya.

4. Jelaskan perbedaan antara module dan package dalam Python. Apa peran file __init__.py dalam sebuah package?
   - **Module** adalah satu file Python dengan ekstensi `.py` yang berisi kode program, seperti fungsi, variabel, dan class. Module digunakan untuk mengorganisasi kode agar dapat digunakan kembali di program lain.
   - **Package** adalah kumpulan module yang dikelompokkan dalam sebuah direktori untuk mengorganisasi kode program agar lebih terstruktur dan mudah dikelola.

     Peran file `__init__.py` dalam package:

   1. Menandai direktori sebagai package regular Python.
   2. Menjalankan kode inisialisasi ketika package diimpor.
   3. Mengatur fungsi, class, atau variabel yang ingin disediakan melalui package.

      Pada Python modern, package juga dapat dibuat tanpa file `__init__.py` melalui mekanisme namespace package. Namun, file tersebut tetap berguna untuk menginisialisasi dan mengatur package.

      Kesimpulannya, Module merupakan satu file Python, sedangkan package merupakan kumpulan module yang dikelompokkan dalam sebuah direktori agar kode lebih terorganisasi.

5. Kapan sebaiknya menggunakan lambda alih-alih def? Sebutkan setidaknya tiga skenario di mana lambda lebih tepat, dan tiga skenario di mana def lebih tepat.

   - lambda dan def sama-sama digunakan untuk membuat fungsi dalam Python, tetapi penggunaannya berbeda sesuai kebutuhan.

   ### A. Tiga skenario penggunaan lambda

   1. Fungsi sederhana: Digunakan ketika fungsi hanya membutuhkan satu ekspresi dan proses yang singkat.
   2. Sebagai argumen fungsi lain: Digunakan ketika membutuhkan fungsi kecil sebagai argumen, misalnya pada map(), filter(), atau sort().
   3. Fungsi sementara: Digunakan ketika fungsi hanya diperlukan untuk satu keperluan tertentu dan tidak perlu didefinisikan secara terpisah.

   ### B. Tiga skenario penggunaan def

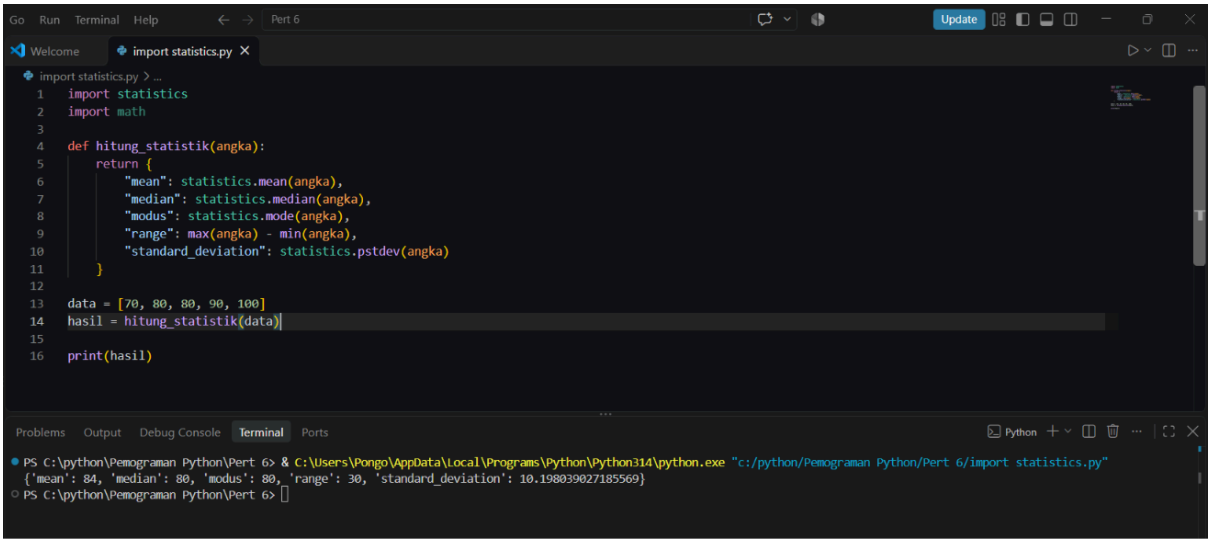
   1. Fungsi kompleks: Digunakan ketika fungsi membutuhkan beberapa langkah proses atau logika yang lebih rumit.

2. Fungsi dengan percabangan dan perulangan: Digunakan ketika fungsi membutuhkan struktur seperti if, else, atau for untuk menjalankan proses tertentu.
3. Fungsi yang digunakan berulang kali: Digunakan ketika fungsi perlu dipanggil berkali-kali, diberi dokumentasi, dan mudah dipelihara.

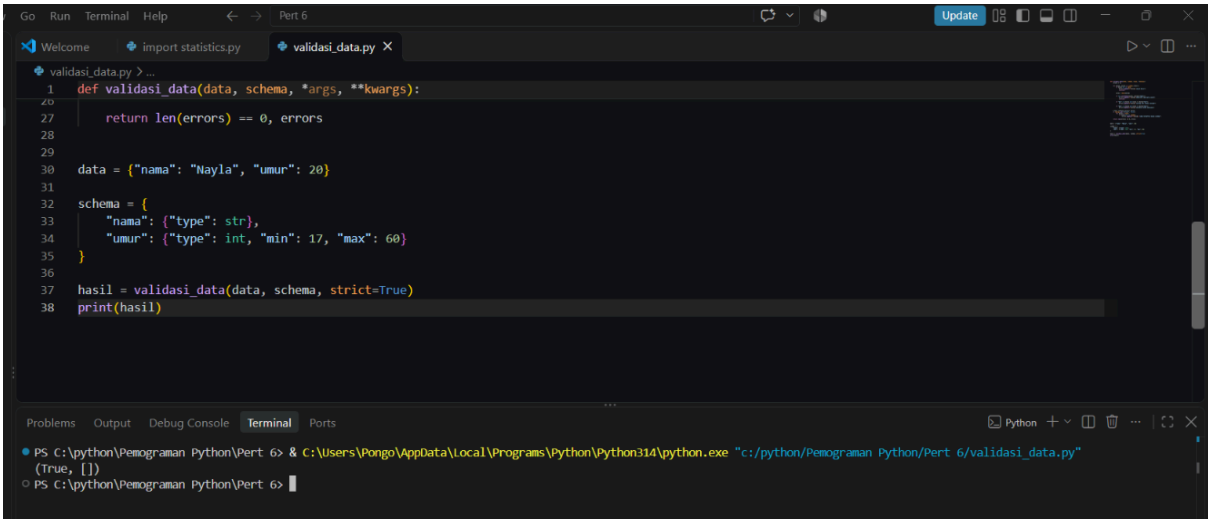
   Kesimpulannya, lambda lebih tepat untuk fungsi sederhana, singkat, dan sementara. Sementara itu, def lebih tepat untuk fungsi yang kompleks, membutuhkan beberapa langkah, atau digunakan berulang kali agar kode lebih mudah dibaca dan dipelihara.

B. Soal Latihan

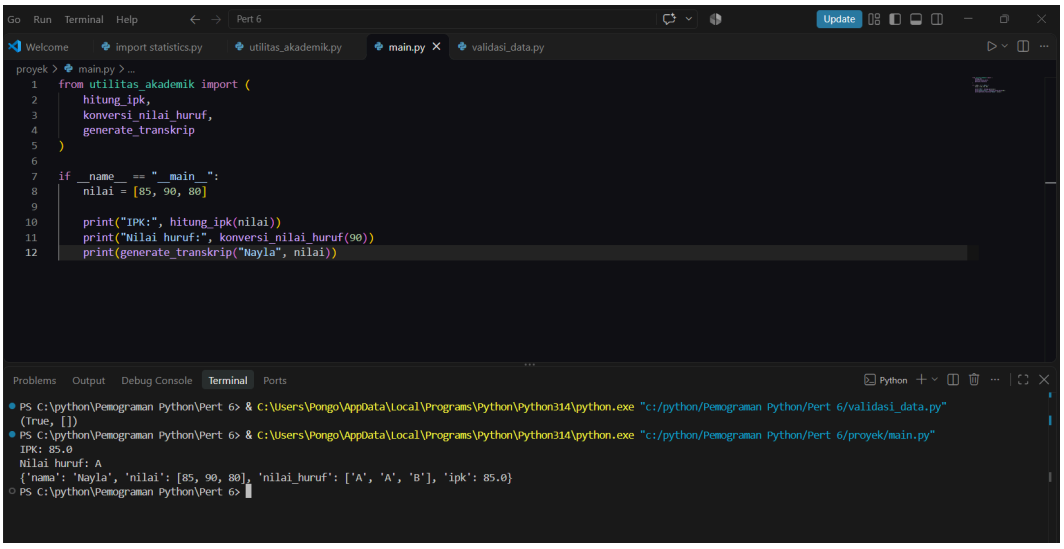
1. Buatlah fungsi hitung_statistik() yang menerima list angka dan mengembalikan dictionary berisi mean, median, modus, range (max-min), dan standard deviation. Gunakan hanya standard library (statistics, math).

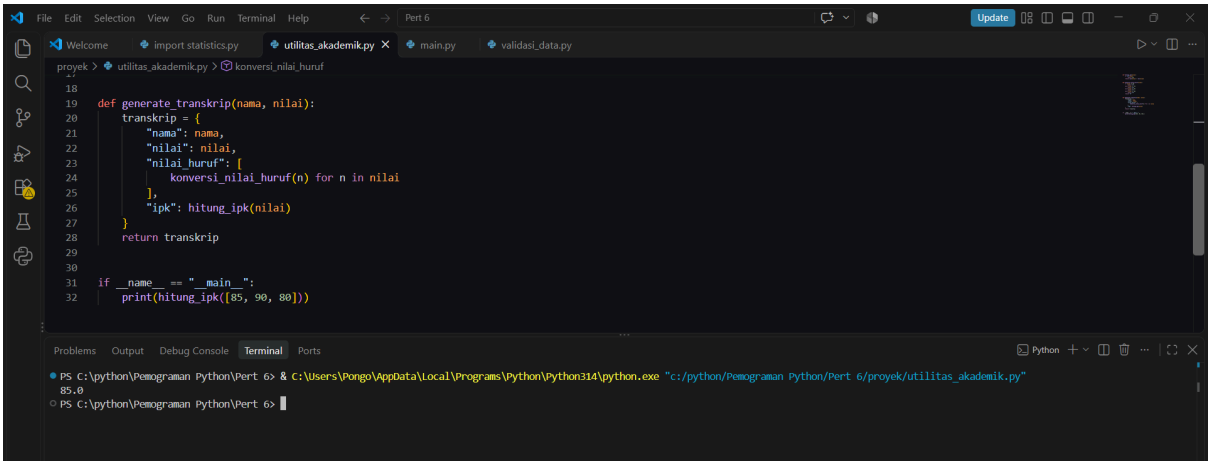


2. Buatlah fungsi validasi_data() yang menerima dictionary data dan schema validasi (berupa dictionary dengan aturan tipe dan rentang). Fungsi mengembalikan tuple (is_valid: bool, errors: list). Gunakan *args dan **kwargs untuk fleksibilitas tambahan.

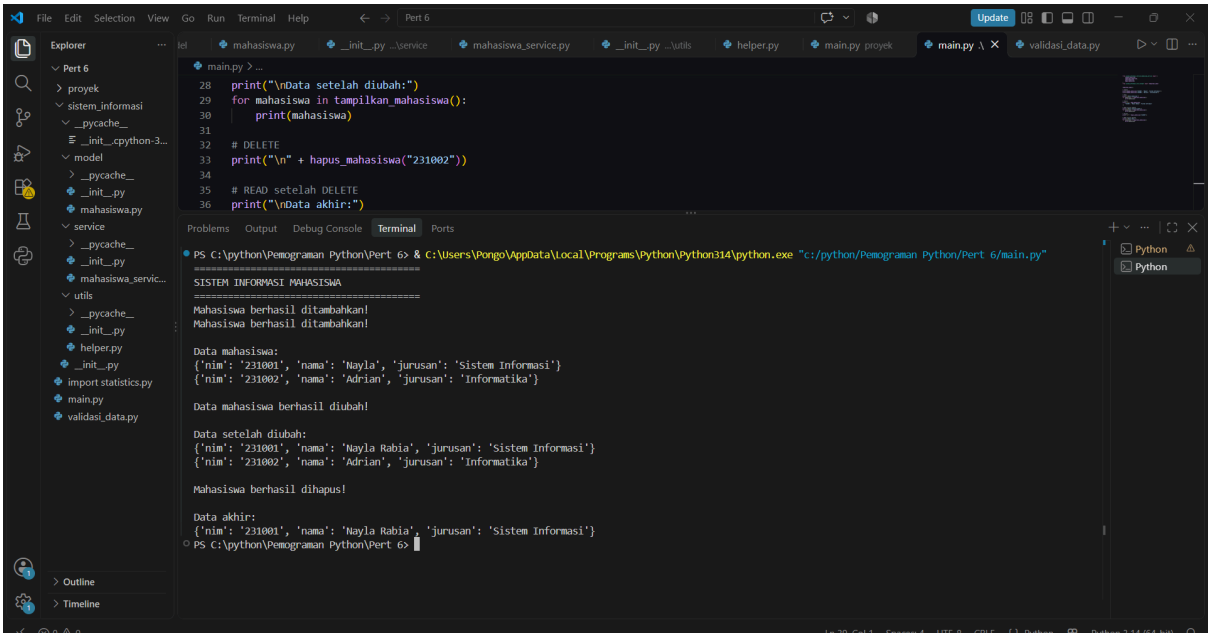


3. Buatlah module utilitas_akademik.py yang berisi fungsi-fungsi: hitung_ipk(), konversi_nilai_huruf(), dan generate_transkrip(). Buat file main.py terpisah yang mengimpor dan menggunakan modul tersebut. Tambahkan if __name__ == '__main__' untuk testing di setiap file.





4. Buatlah package sistem_informasi dengan subpackage: model, service, dan utils. Setiap subpackage harus memiliki __init__.py dan setidaknya satu modul. Implementasikan fungsi CRUD sederhana untuk entitas Mahasiswa menggunakan dictionary sebagai penyimpanan in-memory.



5. Buatlah program yang menggunakan library requests untuk mengambil data dari public REST API (misalnya JSONPlaceholder), kemudian menggunakan lambda dan higher-order function (map, filter, sorted) untuk memproses dan menganalisis data. Simpan hasilnya ke file JSON menggunakan modul json.

